Multi-Drone Search & Rescue for Dynamic Disaster Response Using Graph-Based Path Planning and Priority-Based Task Allocation

***Abstract*—**This paper presents a simulation-oriented coordination system for multiple unmanned aerial vehicles (UAVs) operating in a changing disaster environment. The disaster area is represented as a weighted graph, while rescue requests are maintained using a priority queue so that urgent missions can be considered first. Dijkstra's algorithm is used for weighted shortest-route planning, while breadth-first search (BFS) supports reachability checks. A coordinator combines mission priority, UAV availability, battery level, and route feasibility when selecting a drone. The simulation also models low battery, UAV failure, blocked routes, and newly urgent missions. The implementation uses C for data-structure-oriented logic and C++ classes for object-oriented modelling. Worked scenarios on a six-node example graph show how assignments and route costs change when these events occur.

***Index Terms*—**multi-UAV coordination, search and rescue, graph algorithms, Dijkstra, BFS, priority queue, dynamic task allocation, C, C++

I. INTRODUCTION

Search-and-rescue operations may involve multiple locations, different urgency levels, limited resources, and changing access conditions. A fixed route or first-come-first-served queue is therefore insufficient for the coordination problem considered here. The proposed system treats coordination as a software problem in which missions are prioritised, feasible UAVs are selected, and routes are recalculated when operational conditions change.

The project is deliberately scoped as a simulation rather than an autonomous flight-control system. Its purpose is to connect graph representation, shortest-path algorithms, priority-based task handling, and object-oriented modelling in one explainable workflow.

After an earthquake, flood, or landslide, ground access is often degraded, and aerial platforms are used for surveying affected areas and supporting rescuers [4], [8]. Once more than one UAV is available, three decisions must be made repeatedly: which mission is served first, which UAV serves it, and which route that UAV follows. A plan fixed in advance breaks down when a battery runs low, a UAV fails, a path becomes impassable, or a more urgent request arrives. The contributions of this paper are therefore:

- a weighted-graph model of the disaster zone stored as an adjacency list;
- a priority-queue mission handler that orders requests by urgency;
- a coordinator that combines priority, availability, battery, and route feasibility;
- a controlled dynamic-event module covering four event types; and
- a hand-traceable analytic evaluation on a six-node example graph.

The rest of the paper is organised as follows. Section II reviews related work, Section III states the problem, and Section IV describes the system design. Sections V and VI present the algorithms and the dynamic response, Sections VII and VIII describe the C/C++ implementation and dashboard, and Sections IX–XI provide the evaluation, limitations, and conclusion.

II. RELATED WORK

Previous work has investigated multi-UAV collaboration, persistent surveillance, and coordinated search-and-rescue strategies. LSAR presents collaborative multi-UAV mission planning, while subsequent studies consider persistent surveillance and coordinated UAV search strategies [1]–[3]. These studies motivate coordinated planning; the present project narrows the problem to an undergraduate-friendly algorithmic simulation in which routing, priority-based allocation, and controlled dynamic events can be demonstrated directly.

The system addresses four responsibilities: (1) represent the disaster zone as a weighted graph, (2) maintain rescue missions according to priority, (3) select and route available UAVs, and (4) respond to simulated operational changes. Physical drone hardware, live GPS control, and autonomous victim detection remain outside the implementation scope.

Much of the literature operates at the level of complete frameworks, swarm behaviours, or vehicle platforms, for example the integration of aerial vehicles with a surface carrier in GNSS-denied environments [4]. This paper instead isolates the allocation and routing logic and builds it from classical data structures, so that every decision can be reproduced by hand. A centralised coordinator is adopted because a single decision point is easier to inspect and test in a classroom simulation, at the cost of a single point of failure that a decentralised design would avoid. For route planning, Dijkstra's algorithm [5], [9] is preferred over heuristic searches such as A* [11] because the graph is small, no geometric heuristic is needed, and exact optimality is easy to explain.

The allocation problem also resembles priority scheduling in operating systems, where a queue ordered by urgency decides which job is served next. The difference here is that each served "job" must be matched to one of several limited resources, the UAVs, whose suitability depends on position and battery and therefore changes with every assignment. This is why the priority queue alone is not sufficient and a separate selection step is needed.

III. PROBLEM STATEMENT AND OBJECTIVES

The central problem is to coordinate a small fleet of UAVs when several rescue missions compete for limited resources and the environment may change during execution. The coordinator should favour urgent missions while avoiding assignments that are infeasible because of battery, availability, or route conditions. The objectives are to:

- model the disaster region using a weighted graph and adjacency-list representation;
- prioritise missions using a priority queue;
- select an eligible UAV using availability, battery, and route cost;
- use Dijkstra for weighted route planning and BFS for basic reachability; and
- simulate failures and route changes and re-evaluate affected missions.

Formally, let $G = (V, E, w)$ be an undirected graph with non-negative edge costs w. Each UAV u has a location $\mathrm{loc}(u)$, battery level $\mathrm{bat}(u)$ in percent, and an availability flag. Each mission m has a destination $\mathrm{dest}(m)$ and a priority $\mathrm{prio}(m)$, where a larger value means greater urgency. Let $\delta(x, y)$ be the lowest route cost between nodes x and y, and let $\beta = 10$ be the battery percentage consumed per cost unit. A UAV u is *eligible* for mission m when it is available, $\mathrm{dest}(m)$ is reachable from $\mathrm{loc}(u)$, and $\mathrm{bat}(u) \geq \beta \cdot \delta(\mathrm{loc}(u), \mathrm{dest}(m))$. The coordinator takes missions in non-increasing order of $\mathrm{prio}(m)$ and assigns each to the eligible UAV with the smallest δ, breaking ties in favour of the higher battery level. Because G, the UAV states, and the mission set may all change between decisions, the problem is dynamic.

IV. SYSTEM DESIGN

A. System Architecture

Missions enter through the Mission Input module and are stored in the Priority Queue. The Coordinator repeatedly takes the most urgent pending mission, asks the Route Planner for the cost from each available UAV, and consults the UAV State module for battery and availability. The Event Handler injects operational changes and triggers re-planning. Keeping routing, UAV state, and allocation policy in separate modules lets each be tested and replaced independently.

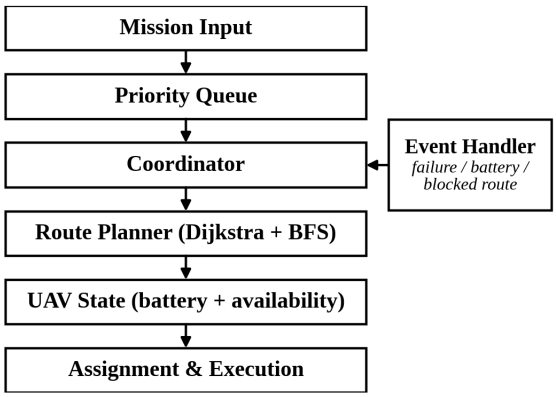


Fig. 1. Vertical coordination architecture with dynamic event feedback to the coordinator.

B. Methodology

The methodology follows a repeatable dynamic-allocation loop. First, the weighted graph, UAV states, battery levels, and pending missions are initialized. Missions are ordered by priority. BFS removes unreachable UAV-mission pairs, while Dijkstra computes the minimum route cost for feasible candidates. Availability and the 10% battery-per-cost rule are checked before selecting the lowest-cost UAV. After assignment, mission and UAV states are updated. Failures, low battery, blocked routes, or urgent requests trigger re-planning and re-allocation for the remaining missions.

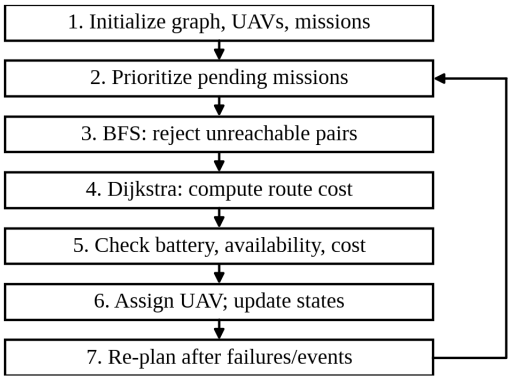
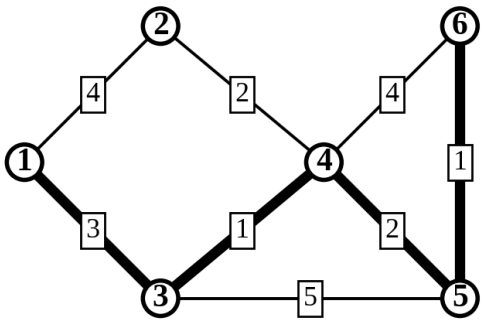


Fig. 1(b). Methodology workflow (dynamic-allocation loop).

C. Graph Representation

Each important location is represented as a node and each traversable connection as an edge. The edge weight represents a simplified travel cost. An adjacency-list representation is used to keep the graph compact and suitable for traversal. The graph contains six nodes and eight weighted connections.



Bold path: 1 → 3 → 4 → 5 → 6; total cost = 7.

Fig. 2. Weighted disaster-zone graph. Edge weights are illustrative route costs, not physical distances.

For the example graph, the route 1 → 3 → 4 → 5 → 6 has total cost 3 + 1 + 2 + 1 = 7. The route comparison in Fig. 3 is derived from the same project graph and is presented as an illustrative algorithmic comparison rather than an experimental benchmark.

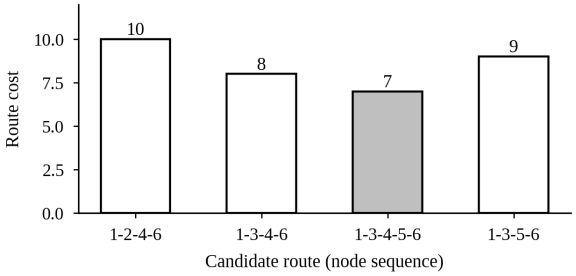


Fig. 3. Illustrative comparison of candidate route costs computed from the project graph. The minimum-cost example route has cost 7.

TABLE I
ADJACENCY LIST OF THE EXAMPLE GRAPH (NEIGHBOUR AND EDGE COST)

Node	Neighbours (cost)
1	2 (4), 3 (3)
2	1 (4), 4 (2)
3	1 (3), 4 (1), 5 (5)
4	2 (2), 3 (1), 5 (2), 6 (4)
5	3 (5), 4 (2), 6 (1)
6	4 (4), 5 (1)

Every undirected edge is stored twice, giving 16 list entries for 8 edges. The list needs space proportional to $V + E$, whereas an adjacency matrix would need V^2 cells. A blocked route is modelled by flagging the edge as impassable so that traversals skip it, without rebuilding the structure.

D. UAV and Mission Models

TABLE II
EXAMPLE UAV STATE AND RESCUE MISSIONS (WITH PRIORITY LEVELS) USED IN THE SIMULATION

UAV	Start	Battery	Example role
UAV-A	Node 1	100%	Primary available unit
UAV-B	Node 3	80%	Secondary available unit
UAV-C	Node 5	60%	Nearby support unit
Mission	**Destination**	**Priority**	**Type**
M-101	Node 6	3	Medical Emergency
M-102	Node 4	2	Person Missing
M-103	Node 5	1	Supply Delivery

A UAV record stores an identifier, current location, battery level, and availability state. A mission stores an identifier, destination, priority, type, and status. The coordinator considers both records when making an allocation decision. In the simplified battery model, route cost is converted into an estimated battery requirement using 10% per route-cost unit; this is a simulation rule and not a physical battery model. Typical states are available, assigned, and failed for a UAV, and pending, assigned, and re-planning for a mission.

E. Modelling Assumptions

- The graph is undirected and all edge costs are non-negative.
- Edge costs are abstract travel costs, not distances or flight times.
- Battery use is linear in route cost, and a UAV is unavailable while assigned.
- One coordinator holds complete, instantly updated information.
- Events are injected deterministically so scenarios can be repeated.

V. ALGORITHMS AND COORDINATION LOGIC

A. Priority-Based Mission Handling

Missions are inserted into a priority queue so that higher-priority rescue requests are considered before lower-priority requests. The queue represents urgency; it does not independently decide which UAV should be assigned. With a binary heap, insertion and extraction both take $O(\log n)$ time [6]. A sequence number can be stored with each mission so that equal-priority missions keep their arrival order.

B. Dijkstra-Based Route Planning

For a weighted graph with non-negative edge costs, Dijkstra's algorithm calculates the lowest-cost route from the current UAV position to the mission destination [5], [9]. The resulting route cost becomes one factor in UAV suitability. Algorithm 1 gives the heap-based form, and Table III traces it from node 1 on the example graph.

Algorithm 1 Dijkstra Route Planning

Input: Graph $G = (V, E, w)$, source node s

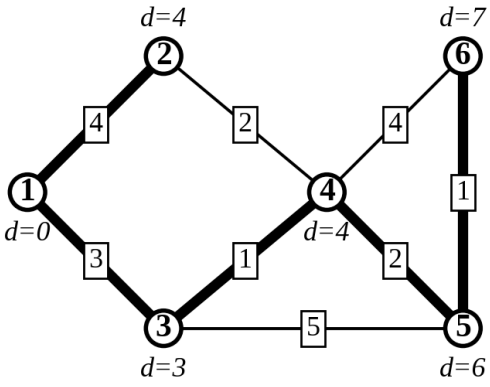
Output: Lowest route cost dist[v] and predecessor prev[v] for every node v

```
dist[v] ← ∞ for every node v; dist[s] ← 0
Push (0, s) into a min-heap H
while H is not empty do
    (d, u) ← extract-min(H)
    if d > dist[u] then
        continue
    end
    for each edge (u, v, w) that is not blocked do
        if dist[u] + w < dist[v] then
            dist[v] ← dist[u] + w; prev[v] ← u
            Push (dist[v], v) into H
        end
    end
end
return dist, prev
```

TABLE III
DIJKSTRA TRACE FROM SOURCE NODE 1 (DISTANCE LABELS AFTER EACH STEP)

Step	Settled	d(2)	d(3)	d(4)	d(5)	d(6)
0	none	∞	∞	∞	∞	∞
1	node 1	4	3	∞	∞	∞
2	node 3	4	3	4	8	∞
3	node 2	4	3	4	8	∞
4	node 4	4	3	4	6	8
5	node 5	4	3	4	6	7
6	node 6	4	3	4	6	7

Following the predecessors back from node 6 gives 6 → 5 → 4 → 3 → 1, that is, the route 1 → 3 → 4 → 5 → 6 with cost 7, matching Fig. 2. Fig. 4 shows the full distance labels and the resulting shortest-path tree.



Shortest-path tree from node 1 (bold); labels show d(v).
Fig. 4. Distances computed by Dijkstra's algorithm from node 1 and the resulting shortest-path tree.

C. BFS Reachability

BFS is used for basic reachability or exploration checks when edge weights are not the primary concern. It can determine whether a destination is reachable before a weighted route is selected. BFS minimises the number of edges, not the cost: from node 1 to node 6 it finds a three-edge route (1 → 3 → 4 → 6 or 1 → 2 → 4 → 6, with costs 8 and 10), whereas the cheapest route has four edges and cost 7. BFS is therefore used only for reachability, and Dijkstra decides the route.

D. UAV Selection

An available UAV is considered eligible when it can support the estimated route cost under the simplified battery rule. Among eligible candidates, route feasibility and current state are considered before assignment. Once selected, the UAV is marked unavailable for the active simulation cycle. Algorithm 2 summarises the coordinator loop.

Algorithm 2 Coordinator Allocation Loop

Input: Mission queue Q (ordered by priority), UAV set U, graph G

Output: Mission-to-UAV assignments

```
while Q is not empty do
    m ← extract the highest-priority mission from Q; best ← none
    for each available UAV u in U do
        if BFS finds no route from loc(u) to dest(m) then
            continue
        end
        c ← Dijkstra cost from loc(u) to dest(m)
        if bat(u) ≥ 10·c and (c, −bat(u)) is lower than best then
            best ← u
        end
    end
    if best exists then
        Assign m to best; mark best unavailable
    else
        Keep m pending for the next cycle
    end
end
```

E. Complexity

TABLE IV
TIME AND SPACE COST OF THE MAIN COMPONENTS (V NODES, E EDGES, k UAVS)

Component	Operation	Time	Space
Adjacency list	build / traverse	O(V + E)	O(V + E)
Priority queue	insert / extract	O(log n)	O(n)
Dijkstra (heap)	one source	O((V + E) log V)	O(V)
BFS	one source	O(V + E)	O(V)
UAV selection	one mission	O(k (V + E) log V)	O(V)

The selection cost in Table IV assumes one Dijkstra run per candidate UAV [10]. Because the graph is undirected, a single run started at the mission destination would give the cost for every UAV at once, which is a simple optimisation for larger fleets. For the six-node example all of these costs are negligible.

F. Algorithm Choice

Each method answers a different question. BFS (fewest edges) answers whether a destination can be reached at all, Dijkstra [5], [9] (lowest cost, non-negative weights) answers how cheaply, and A* [11] answers the same question faster when node coordinates make a good heuristic available; it is kept for future use. Using BFS first lets the coordinator reject an unreachable mission immediately instead of running a full weighted search that cannot succeed.

VI. DYNAMIC RESPONSE

The simulator introduces four representative events: low battery, UAV failure, blocked route, and an urgent mission. The coordinator checks the affected mission state and updates the route or assignment when necessary. Algorithm 3 shows how it responds to each event, and Fig. 5 shows the order of the re-evaluation steps.

Algorithm 3 Dynamic Event Handler

Input: Event e, graph G, UAV set U, mission queue Q

Output: Updated assignments after the event

```
1  if e is LowBattery(u) then
2    if bat(u) < 10·cost(m(u)) then
3      Release m(u); push m(u) into Q
4    end
5  else if e is UAVFailure(u) then
6    state(u) ← failed
7    Push m(u) into Q at its original priority
8  else if e is BlockedRoute(a, b) then
9    blocked(a, b) ← true
10   for each assigned mission m do
11     Re-run Dijkstra and BFS for m on the updated graph
12   end
13 else if e is UrgentMission(m′) then
14   Push m′ into Q
15 end
16 Run Algorithm 2 on Q; unserved missions stay pending
```

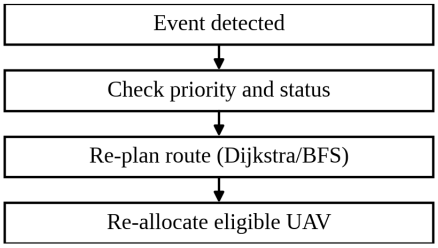


Fig. 5. Dynamic-response sequence used when an operational event changes route or UAV feasibility.

Re-evaluation follows a fixed order: the event is applied to the graph or UAV state, the affected missions are identified, their priority and status are checked, and only then are routes and assignments recomputed. A mission that no UAV can serve stays pending rather than being dropped, and it is retried in the next cycle.

A blocked route is the clearest case of why costs must be recomputed rather than reused. When edge 4–5 is flagged as impassable, only the route 1-3-4-5-6 from Fig. 3 changes: it becomes unusable, while the other three keep their costs of 10, 8, and 9. Because the unusable route was the cheapest one, the best available cost rises from 7 to 8. A BFS run on the updated graph still reaches node 6, so the mission remains feasible. A battery or priority decision that relied on the old cost of 7 would now be wrong, which is why the coordinator re-runs Dijkstra on every affected mission instead of caching routes.

Events can also interact. A low-battery report and a newly inserted urgent mission may arrive in the same cycle, in which case the urgent mission is placed in the queue first and eligibility is then checked against the updated battery levels, so that a weakened UAV is never assigned on stale information.

VII. C/C++ IMPLEMENTATION

The implementation uses C for data-structure-oriented logic and C++ for object-oriented modelling. The graph uses an adjacency-list structure, the mission queue uses priority ordering, and UAV and Mission classes encapsulate state. A Coordinator class connects these modules and exposes operations such as mission processing, event handling, low-battery simulation, and failure simulation. C++ classes are user-defined types [7], [12], while priority handling can be implemented with the standard priority_queue abstraction [6], [7].

```
/* C: adjacency-list graph */
typedef struct Edge {
    int to, cost, blocked;
    struct Edge *next;
} Edge;
typedef struct { int n; Edge *adj[MAXN]; } Graph;

// C++: state and coordination
class UAV { int id, node, battery; bool available; };
class Mission { int id, dest, priority;
                std::string type, status; };
class Coordinator {
    void processMissions();           // Algorithm 2
    void handleEvent(Event e);        // Algorithm 3
    void simulateLowBattery(int id);  // event 1
    void simulateFailure(int id);     // event 2
};
```

Listing 1. Representative declarations following the structure described above (illustrative excerpt).

One processing cycle has five steps: (1) read or insert missions; (2) extract the highest-priority mission; (3) compute the route cost for every available UAV; (4) apply the eligibility rule and assign the best candidate; and (5) report the assignment and update UAV state. Event simulation operates between cycles by changing the graph or UAV records and then re-running the affected steps.

Because every scenario is deterministic, the console output of a run can be compared line by line with the hand-derived assignments reported in Section IX.

VIII. VISUALISATION AND DASHBOARD

A lightweight dashboard is proposed as the visual command-centre layer. It does not replace the C/C++ implementation; instead, it exposes the same simulation concepts in a form that makes UAV status, mission priority, routes, and dynamic events easier to inspect. The dashboard is organised around overview, UAV fleet, mission queue, route visualisation, event simulation, and activity-log views.

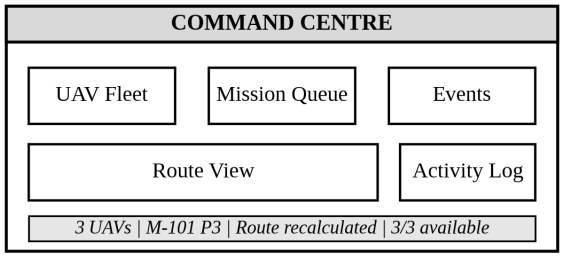


Fig. 6. Original black-and-white dashboard concept redrawn for this paper; it is a visual prototype, not a copied interface.

The UAV fleet view lists location, battery, and availability for each drone, and the mission queue view lists pending missions in priority order. The route view highlights the active path on the graph, the event view triggers the four scenarios, and the activity log records each assignment and recalculation in time order, so that a reviewer can follow how a decision was reached.

IX. EVALUATION AND DISCUSSION

A. Evaluation Approach

The prototype should be evaluated primarily for correctness and explainability rather than real-world flight performance. Repeatable scenarios include normal allocation, low-battery UAV, UAV failure, blocked route, and urgent mission insertion. Useful software-level measures are assignment success rate, route cost, number of successful re-plans, response to failures, and simulation processing time. No measured benchmark values are claimed here because the scenarios are evaluated analytically and no experimental runs are reported.

B. Analytic Walk-Through

To show the expected behaviour, the allocation rule of Section III was applied by hand to the missions and UAVs of Table II. Fig. 7 gives the battery each UAV would need for each mission under the 10% rule. In the normal case, M-101 (priority 3) is served first: UAV-A, UAV-B, and UAV-C would need 70%, 40%, and 10%, all within their batteries, so UAV-C is chosen as the lowest-cost eligible unit. M-102 then goes to UAV-B (cost 1), and M-103 goes to UAV-A (cost 6), giving a total route cost of 8. Among the six possible one-to-one assignments the minimum total is also 8, but the greedy rule offers no such guarantee in general.

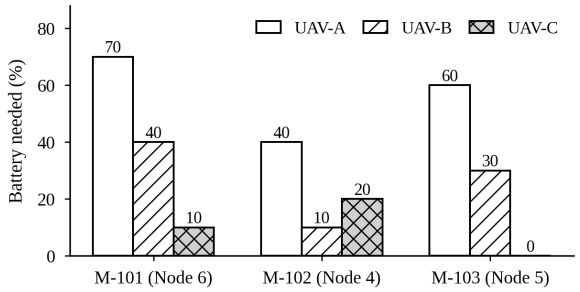


Fig. 7. Battery percentage each UAV would need for each example mission under the simulation rule (analytic values).

TABLE V
ANALYTIC OUTCOME OF EACH SCENARIO (MISSION → UAV, ROUTE COST)

Scenario	Assignments	Served	Total cost
Normal	M-101→C (1), M-102→B (1), M-103→A (6)	3 of 3	8
Low battery (C at 5%)	M-101→B (4), M-102→A (4), M-103→C (0)	3 of 3	8
UAV-C failure	M-101→B (4), M-102→A (4), M-103 pending	2 of 3	8
Edge 4–5 blocked	M-101→C (1), M-102→B (1), M-103→A (8)	3 of 3	10
Urgent M-104 (P4, node 2)	M-104→B (3), M-101→C (1), M-102→A (4), M-103 pending	3 of 4	8

Table V shows how each event changes the outcome. When UAV-C's battery falls to 5%, it can no longer cover the 10% needed for M-101, so UAV-B takes that mission. When UAV-C fails, there are only two UAVs left and the lowest-priority mission, M-103, stays pending. Blocking edge 4–5 raises the best route from node 1 to node 6 from 7 to 8 (an increase of about 14%) and the route from node 1 to node 5 from 6 to 8, so M-103 now needs 80% battery from UAV-A, which it can still supply. Inserting M-104 with priority 4 places it ahead of every other request; with only three UAVs, M-103 is again left pending. The resulting assignment success rates are 100%, 100%, 67%, 100%, and 75%.

All values in this subsection follow from the model and are not measurements of an executed program.

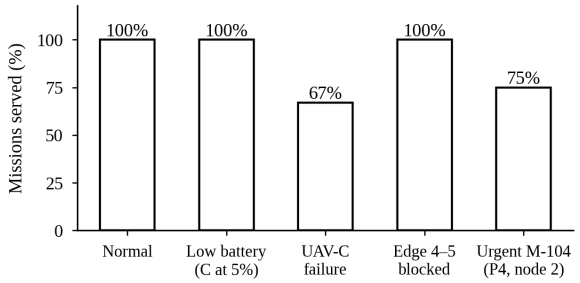


Fig. 8. Analytic assignment success rate for each scenario of Table V.

C. Verification

Each scenario can be checked by running the simulator and comparing the printed output with Table V. In the normal case all three missions must be assigned as listed; in the low-battery case UAV-C must be rejected for M-101 and replaced by UAV-B; after a UAV failure M-103 must stay pending with nothing assigned to the failed UAV; when edge 4–5 is blocked the cost to node 6 must rise from 7 to 8 while BFS still reaches it; and after the urgent insertion M-104 must be served first with M-103 left pending. Running the five scenarios and comparing the printed output with these expectations gives a pass or fail result for each behaviour, which is a practical way to demonstrate correctness in a project review.

D. Discussion

The results show three properties of the design. First, priority ordering is protective for urgent missions but can starve low-priority ones whenever capacity is short, as M-103 shows. Second, a greedy per-mission choice is simple and traceable but not globally optimal in general, so an assignment-problem formulation would be a natural comparison. Third, the centralised coordinator makes every decision inspectable, but it is also a single point of failure and relies on accurate state information. These observations apply to the simplified model and should not be read as claims about field performance.

A representative execution sequence is: missions enter the priority queue; the coordinator selects a feasible UAV; Dijkstra provides the lowest-cost route; and a simulated event triggers route recalculation or reassignment. This sequence demonstrates the interaction among graph data structures, priority queues, algorithms, and C/C++ object models.

X. LIMITATIONS AND FUTURE SCOPE

The current model simplifies battery consumption, mission completion, and flight behaviour. It does not model physical flight dynamics, GPS uncertainty, weather, communication loss, or autonomous victim detection. The coordinator also treats a selected UAV as unavailable during the active mission cycle rather than continuously simulating flight.

Future extensions can include real-time map updates, more realistic battery estimation, automatic reassignment after failed missions, mission-completion and return-to-base logic, richer dashboard interaction, and eventual integration with a robotics or drone simulator. Larger maps with coordinates could use heuristic search such as A* [11], and the greedy allocator could be compared with an optimal assignment method or a decentralised scheme. The hand-derived scenarios could also be complemented by randomly generated graphs of increasing size, measuring simulation processing time and assignment success rate as the number of missions per UAV grows.

Any step from simulation towards real deployment would also need to address airspace regulation, operator oversight, and data privacy. A coordinator of this kind is a decision-support tool, and the final authority over rescue flights should remain with trained human operators.

XI. RESULTS AND CONCLUSION

A. Results

The five scenarios of Table V were evaluated analytically with the rule of Section III; all values are model-derived, not measured. Fig. 9 splits each scenario's route cost by mission: four scenarios total 8, whereas blocking edge 4–5 raises it to 10 because M-103 is now served at cost 8 instead of 6.

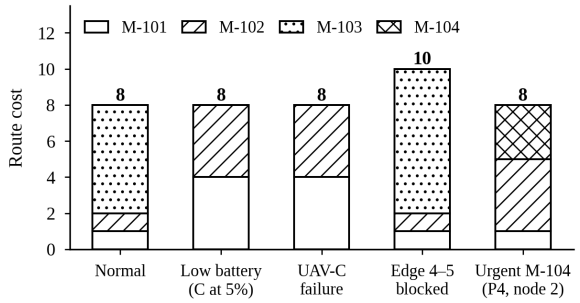


Fig. 9. Route cost per Table V scenario, stacked by mission (analytic).

Fig. 10 shows that all missions are served in three scenarios, while the UAV-C failure (2 of 3, 67%) and the urgent M-104 insertion (3 of 4, 75%) each leave M-103 pending: route cost rose only when the graph changed, and coverage dropped only when UAVs were fewer than requests.

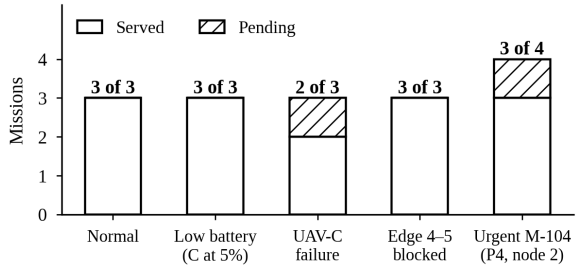


Fig. 10. Missions served and pending per Table V scenario (analytic).

B. Conclusion

The proposed multi-drone search-and-rescue system combines graph-based path planning with priority-based task allocation to demonstrate a dynamic coordination problem. Dijkstra provides weighted route planning, BFS supports reachability, the priority queue represents mission urgency, and C/C++ data structures and classes provide the implementation foundation. Controlled failure and route-change events demonstrate why a rescue coordinator must reconsider both missions and available UAVs during execution.

The worked scenarios make the need for dynamic re-planning concrete. A single blocked edge raised the best route cost by about 14%, a single failed UAV left one of three missions unserved, and one urgent request displaced the lowest-priority mission. None of these outcomes could be produced by a fixed route or a first-come-first-served queue, which supports the design choice of recomputing routes and eligibility whenever the situation changes.

The resulting system is intentionally designed as a simulation and visualisation prototype. Its value lies in connecting Data Structures, algorithms, and OOP concepts to one coherent application while leaving a clear path for future expansion.

REFERENCES

[1] E. T. Alotaibi, S. S. Alqefari, and A. Koubaa, “LSAR: Multi-UAV collaboration for search and rescue missions,” *IEEE Access*, vol. 7, pp. 55817–55832, 2019, doi: 10.1109/ACCESS.2019.2912306.

[2] H. Gharrad, A. U. H. Yasar, and W. Ectors, “Simulating collaborative and autonomous persistent surveillance by drones for search and rescue operations,” *Transportation Research Procedia*, vol. 84, pp. 209–218, 2025, doi: 10.1016/j.trpro.2025.03.065.

[3] A. A. Kareem *et al.*, “Enhancing search and rescue missions for victim detection using a coordinated bio-inspired dual-altitude UAV swarm framework,” *Engineering Science and Technology, an International Journal*, vol. 77, Art. no. 102353, 2026, doi: 10.1016/j.jestch.2026.02.002.

[4] Y. Dong *et al.*, “Drone carrier: An integrated unmanned surface vehicle for autonomous inspection and intervention in GNSS-denied maritime environment,” *Journal of Ocean Engineering and Science*, vol. 11, no. 4, pp. 1271–1286, 2026, doi: 10.1016/j.joes.2026.02.002.

[5] GeeksforGeeks, “Dijkstra’s Algorithm,” 2026. [Online]. Available: https://www.geeksforgeeks.org/dsa/dijkstras-shortest-path-algorithm-greedy-algo-7/

[6] GeeksforGeeks, “Priority Queue in C++ STL,” 2026. [Online]. Available: https://www.geeksforgeeks.org/cpp/priority-queue-in-cpp-stl/

[7] cppreference.com, “Classes,” 2026. [Online]. Available: https://en.cppreference.com/cpp/language/classes

[8] J. Lea/DFID, “Drone surveillance helps search and rescue in Nepal,” Wikimedia Commons, Apr. 29, 2015, CC BY 2.0. [Online]. Available: https://commons.wikimedia.org/wiki/File:Drone_surveillance_helps_search_and_rescue_in_Nepal_(17106234127).jpg

[9] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959.

[10] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

[11] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Trans. Syst. Sci. Cybern.*, vol. 4, no. 2, pp. 100–107, 1968.

[12] B. Stroustrup, *The C++ Programming Language*, 4th ed. Upper Saddle River, NJ, USA: Addison-Wesley, 2013.